

PayMind

A scalable tool-calling agent for PayPal

Users ask in plain English. The agent finds the right PayPal API among many, fills in the parameters, lets **code** check the call, asks the user before anything changes, runs it, and shows what really happened. Every step is traced in LangSmith.

Akshayaa D · github.com/akshayaad19/paymind · September 2026

What is PayMind?

PayMind is a **seller's portal with an AI assistant** for a shop that takes payments through PayPal. Instead of clicking through PayPal's many screens and APIs, the shop's staff and its customers simply **ask in plain English**: *"send an invoice for \$50 to john@x.com", "what were my sales last month?", "I want a refund for my headphones"*. The assistant finds the right PayPal action, fills in the details, asks for confirmation when something will change, does it, and shows what really happened.

The accountant (Asha) sees the whole shop: sales and balance, invoices, disputes and purchase orders. She refunds or replaces, chases overdue invoices and ships orders, from the chat or the screens.

Customers (Rahul, Priya) see only their own purchases, invoices and cases. They pay invoices, send purchase orders (even handwritten), report problems, attach photos, ask for a refund or replacement, and close their own cases.

Behind the scenes it is built for scale: PayPal alone has over a hundred APIs and a real business may have thousands, so the assistant never sees them all. For each message it **searches** for the few most relevant tools; **code** checks every action, the user **confirms** anything that changes data, and **every step is traced** in LangSmith.

112

PayPal APIs as searchable tool cards, from the Postman collection

100%

right tool in the top 5 at **1,012 tools** (with 900 look-alike distractors)

~20 ms

per tool search (hybrid BM25 + dense, RRF)

236

automated tests, run without any LLM

The idea in one line. The model never sees every tool: each message runs a **role-filtered hybrid search** and offers only the **top 5** tools plus a few built-ins (including the **RAG pipeline tool** and the **System Search tool**). The prompt stays the same size whether there are 100 or 10,000 tools, and everything with a right answer (permissions, amounts, IDs, confirmation) is decided by code, not by the model.

Contents

1. What the brief asked for
2. Tech stack
3. System architecture
4. Life of one message
5. Agent structure
6. Tool selection and routing
7. RAG pipeline and System Search
8. State and memory

9. Error handling and safety
10. Observability: what the traces show
11. Scalability: measured

12. Scaling to thousands of customers
13. Why these choices
14. Limitations and next steps

1 What the brief asked for, and where it is

Requirement	In PayMind	Section
Agentic system over the PayPal Postman collection (50+ APIs)	112 requests parsed and enriched into tool cards; a mock PayPal server answers them (30 endpoints with real state in SQLite; the rest replay PayPal's own example responses).	3, 6
"Send an invoice for \$50 to..."	<code>create_draft_invoice</code> → <code>send_invoice</code> , each confirmed by the user.	4, 5
"Total sales volume last month?"	<code>list_transactions</code> for the date range; the model adds up the sales.	6
"Is there a dispute open from user_123?"	<code>list_disputes</code> ; results filtered to what the caller may see.	6, 9
RAG Pipeline Tool	<code>rag_search</code> over 15 policy documents → 336 chunks → hybrid search + cross-encoder reranker; answers only from passages, with citations.	7
System Search Tool	<code>system_search</code> (capabilities + the user's own activity) and <code>find_tools</code> (whole catalogue).	7
Scale from 100+ to 1000s of tools	Retrieval-first tool selection, measured at 112 / 512 / 1,012 tools.	6, 11
Agent structure · routing · state · scalability · error handling	One LangGraph ReAct agent with code nodes around the LLM.	5, 6, 8, 9, 12
Framework choice · observability	LangGraph (justified) · LangSmith traces with custom spans.	10, 13

Beyond the brief, the demo is a working seller's portal for one shop: an accountant and customers log in, chat with the agent, handle disputes end to end (messages, photos, refund or replacement, shipment tracking; only the customer closes a case), chase overdue invoices, and turn handwritten purchase orders into invoices and deliveries.

2 Tech stack

Layer	Technology	Used for
LLM	Google Gemini (3.8 Flash, falling back to 3.6 / 3.5 Flash / Flash-Lite) via <code>langchain-google-genai</code>	Choosing and filling tool calls, answers, writing messages; multimodal reading of handwritten purchase orders into a Pydantic form. Also the offline enricher.
Agent	LangGraph <code>StateGraph</code> , <code>interrupt()</code> , <code>stream_mode="messages"</code> ; <code>langchain-core</code> <code>messages</code>	The ReAct loop as 4 nodes, pausing for human approval, streaming the answer token by token.
State	LangGraph SQLiteSaver <code>checkpointer</code>	Agent state and chat memory per user and session; resume after a Yes/No.
Vector DB	Qdrant Cloud (free tier); in-memory Qdrant for tests and evaluation	Two collections: <code>tools</code> (112) and <code>docs</code> (336). Sparse + dense vectors, server-side RRF, payload indexes and filters (role).
Embeddings	fastembed (ONNX, local): <code>BAAI/bge-small-en-v1.5</code> (384-d, cosine) and <code>Qdrant/bm25</code> (sparse, IDF)	Meaning search and keyword search, computed locally: no embedding API cost.
Reranker	Cross-encoder <code>Xenova/ms-marco-MiniLM-L-12-v2</code> (<code>fastembed</code>)	Re-orders the top 20 policy chunks by reading question and chunk together.
Ingestion	Postman collection v2.1 parser (own code) · <code>langchain-text-splitters</code> · <code>trafilatura</code>	API cards from Postman; policy pages fetched as clean Markdown, split by heading then 2,000 characters.
Validation	Pydantic v2 , <code>jsonschema</code>	Enricher output, tool parameters (JSON Schema), request bodies, PO extraction.
API	FastAPI + Uvicorn, Server-Sent Events	REST endpoints for the web app; streamed chat answers.
Auth	PyJWT (HS256), <code>hashlib.scrypt</code>	Login tokens with a version (logout revokes copies); salted password hashes; role read from the database on every request.
PayPal calls	httpx executor	Builds path / query / body from each card, retries with backoff, reuses the <code>PayPal-Request-Id</code> (idempotency).
PayPal (mock)	FastAPI + SQLite	Real request/response shapes; one transaction per request; idempotency replay; failure injection for tests.
App data	SQLite	Users, audit log, purchase orders, refund/replacement requests, dispute photos, read markers.
Front end	Plain HTML / CSS / JavaScript	Chat (streaming reader), account data with customer filter, orders, audit log; light and dark mode.
Documents	<code>fpdf2</code>	Invoice PDFs.

Observability	LangSmith (<code>@traceable</code> spans + LangGraph auto-tracing)	One trace per message; custom spans for tool search, validation, PayPal calls and RAG (section 10).
Testing	pytest , FastAPI <code>TestClient</code> , <code>httpx.MockTransport</code> , a scripted fake LLM	236 tests across parser, enricher, index, mock, executor, validator, agent graph, API and RAG, with no LLM calls.

3 System architecture

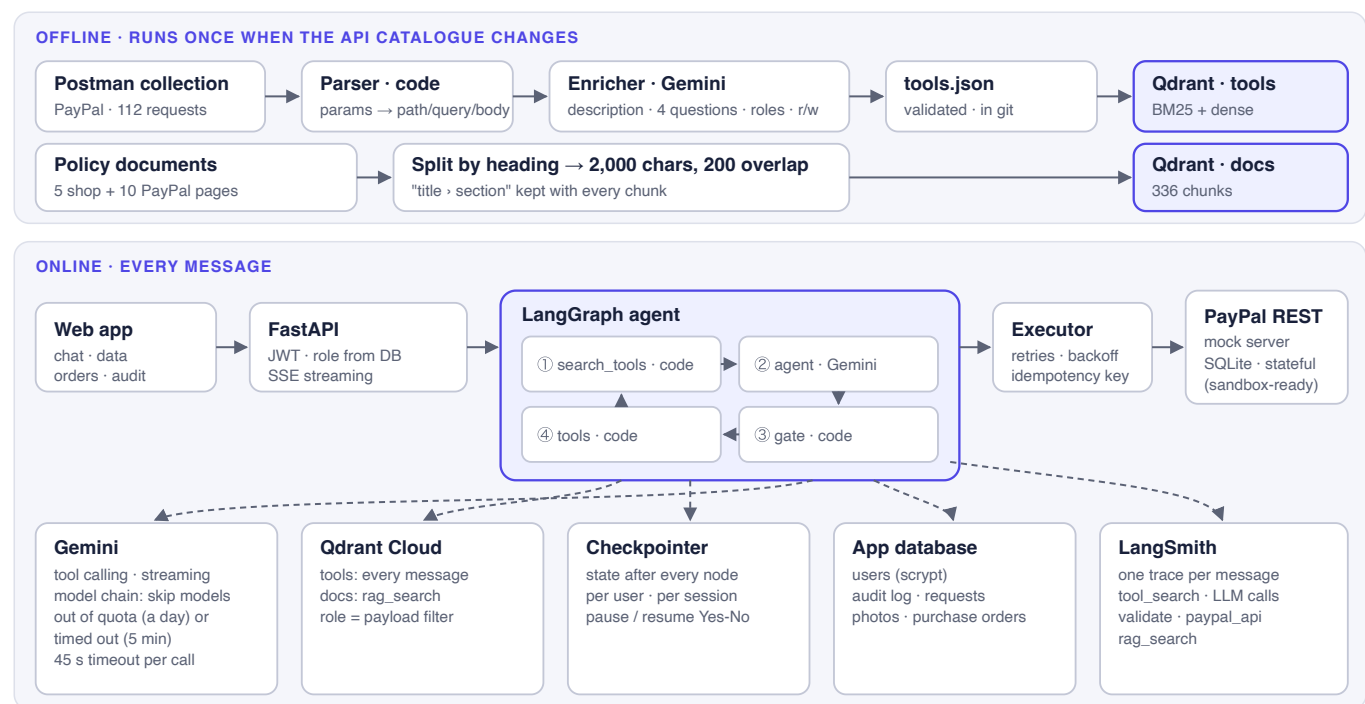


Figure 1. Offline, the Postman collection becomes validated, searchable tool cards (the enricher's output is schema-checked and every example call is dry-run against the mock). Online, only code talks to PayPal: the model proposes, the gate checks, the executor runs.

Component	Responsibility
Parser + enricher	Turn each Postman request into a card: name, method, path, parameters (with where each goes), then add a plain description, 4 example questions, allowed roles, read/write, required fields and a validated example call.
Tool registry	<code>tools.json</code> : the single source of truth for what a tool is, who may use it and how to call it. Loaded into memory at start.
FastAPI app	Login, streaming chat, account data, disputes, purchase orders, invoices. Every endpoint checks the role on the server.
LangGraph agent	The loop in section 5. Only node ② is an LLM.
Executor	The only thing that calls PayPal: builds the HTTP request from the card, retries safely, returns one plain sentence on errors.

4 Life of one message

"Send an invoice for \$50 to john@x.com", from the accountant:

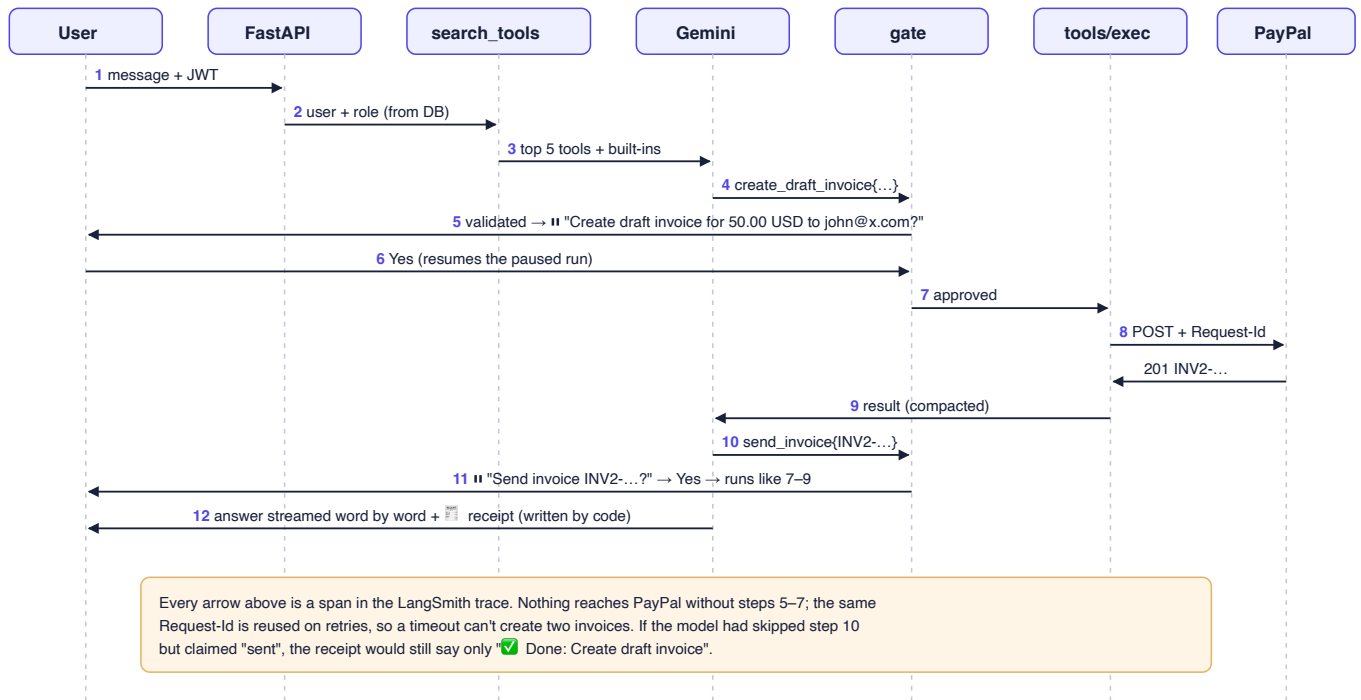
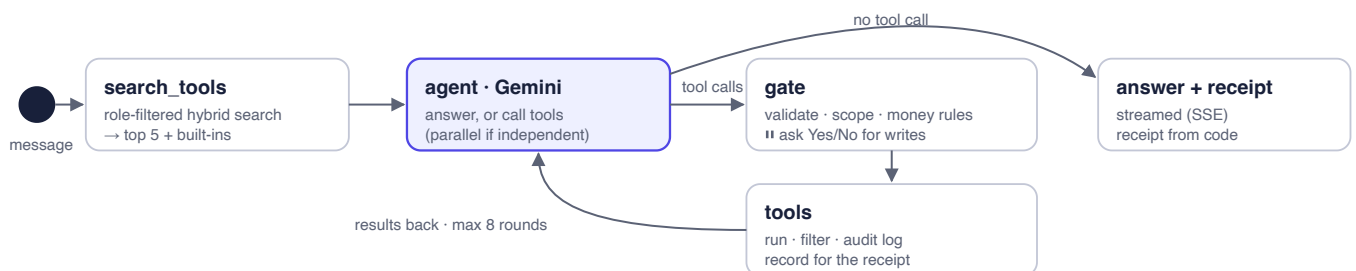


Figure 2. The two writes each pause the graph for the user's approval; the run resumes from the saved state in a new HTTP request.

5 Agent structure

One **ReAct** agent (reason → act → observe, repeated) as a LangGraph state machine. The loop runs as many rounds as the question needs (0 for "what can you do?", 3 for "refund Rahul's disputed payment"), capped at 8.



Node	LLM?	Job
search_tools	No	Picks this message's tools (section 6). Short follow-ups ("refund the first one") are searched with the previous message.
agent	Yes	Reads the conversation, tool results and errors; calls tools (chaining dependent ones, e.g. list → details → refund) or answers.
gate	No	Checks every proposed call (section 9); for writes, pauses with <code>interrupt()</code> . Changes no data, so a resumed run never repeats an action.
tools	No	Runs approved calls, filters list results to the caller's own records, writes the audit log, records what really ran.

Why one agent, not several? Tool search already does the routing a supervisor would do, for one LLM call fewer per message, and safety stays in deterministic code rather than a second "validator" LLM. Domain sub-agents can be added as LangGraph sub-graphs if one prompt can no longer hold the rules (section 12).

6 Tool selection and routing

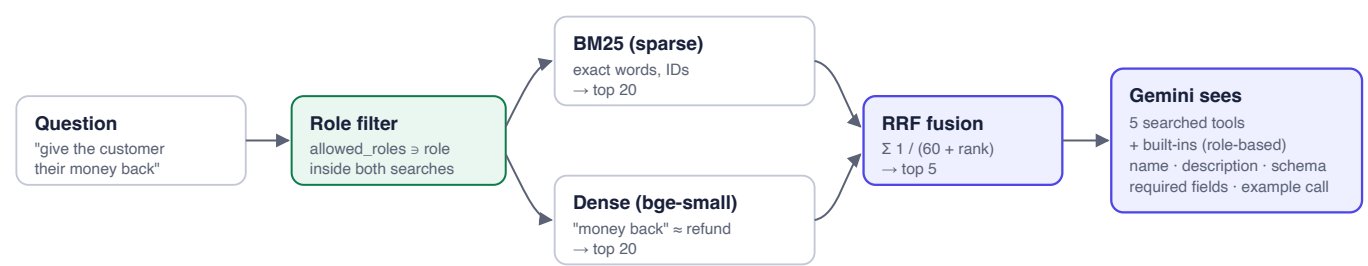


Figure 3. Retrieval-first routing. Each card is indexed from its name, category, description and four example questions.

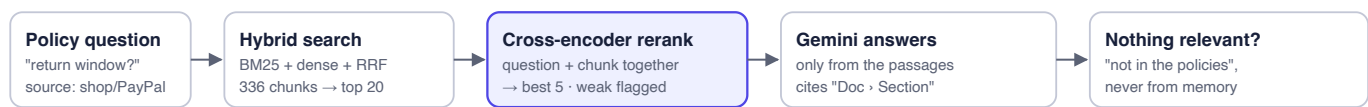
Built-in tools (always offered)

Tool	For
<code>rag_search</code>	Policy questions (RAG tool)
<code>system_search</code>	What can you do · my activity
<code>find_tools</code>	Search the full catalogue
<code>check_updates</code>	What needs my attention
<code>order_status</code>	Purchase orders, tracking
<i>customer only</i>	<code>my_purchases</code> , <code>report_problem</code> , <code>request_resolution</code> , <code>close_case</code>
<i>shop only</i>	<code>customer_payments</code> (find the right customer among many)

Why this scales

- The prompt holds 5 tools whatever the catalogue size: cost and accuracy don't degrade as APIs are added.
- Adding an API = add a card and re-index. No prompt changes, no code changes.
- Permissions are enforced in the search itself: a customer's model never sees a refund tool.
- `find_tools` lets the model recover if the first 5 miss.
- Measured at 1,012 tools in section 11.

7 RAG pipeline tool and System Search tool



System Search answers questions about the assistant itself: `mode=capabilities` searches the tool index (role-filtered, synthetic test tools hidden) for "what can you do with invoices?", and `mode=activity` reads the user's own audit log for "what's the status of my last request?". Where policy involves money it is also **enforced in code**: e.g. in-store purchases can get at most 50% back, whatever the model says.

8 State and memory

AgentState field	Holds
messages	The conversation (appended)
offered	Tools the model may call
seen	Text used to check IDs are real
steps	Rounds so far (max 8)
decisions	The gate's verdict per call
actions	What really ran → receipt

- **One shared state**; each node returns only the fields it changes (`messages` is appended, the rest replaced).
- The **checkpointer** saves it after every node, per `user_id__session` : chat memory for follow-ups, and a paused run resumes exactly where it stopped after the user's Yes.
- **Identity**: the JWT only says who you are; the role is read from the database on each request, and a token version lets logout revoke every copy.
- **Business data**: PayPal is the source of truth; the app database keeps only what PayPal has no concept of.

9 Error handling and safety

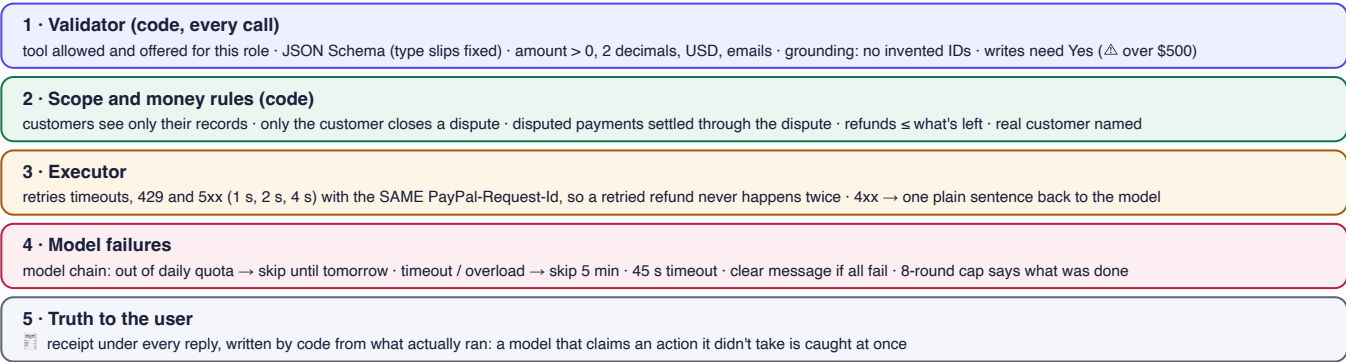


Figure 4. Five layers. Layers 1–2 prevent; 3–4 recover; 5 tells the user the truth. Errors from layer 1 go back to the model to fix.

10 Observability: what the traces show

Every chat message is **one LangSmith trace** named `paymind_chat` . LangGraph traces each node automatically; four **custom spans** (`@traceable`) record our own decisions. Filter by the tag `role:customer` / `role:accountant` , or by user and session in metadata.



Figure 5. One trace. Green spans are our custom spans: the code decisions that make the system safe, and the first place to look when something goes wrong.

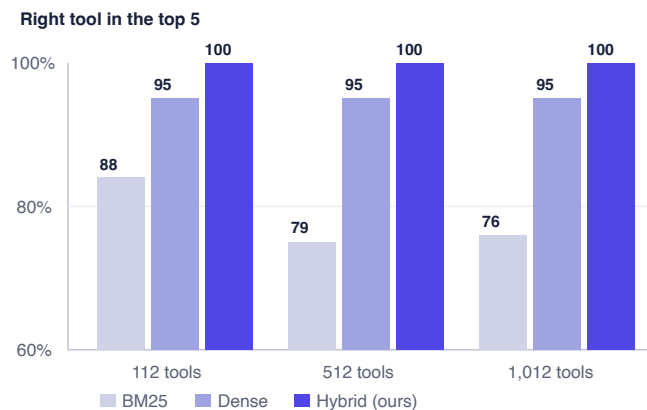
Symptom	Where the trace shows the cause
Wrong or no tool used	<code>tool_search</code> : was it in the top 5? If yes, the LLM span shows what it chose instead.
"It said done but nothing happened"	<code>validate</code> (blocked / invalid?) and whether a <code>paypal_api</code> span exists; the receipt shows the same.
Slow answer	Per-span latency: nearly always the LLM (or a fallback after a timeout); PayPal calls take milliseconds.
Access question	<code>paypal_api</code> records the caller, the tool's allowed roles and <code>role_allowed</code> .

For users, the in-app **audit log** keeps every action (who, what, when, result); developer detail stays in LangSmith.

11 Scalability: measured

Does tool search still find the right tool at 1,000+ tools? 42 hand-written questions (not from the tool cards) were searched over the 112 real tools, then with 400 and 900 synthetic tools from 25 look-alike services (other payment, billing, CRM, accounting, shipping and support APIs) that reuse PayPal's vocabulary: "refund a charge", "list invoices", "respond to dispute".

Tools	Search	Top-5	Top-1	ms
112	BM25	88%	60%	10
112	Dense	95%	83%	10
112	Hybrid	100%	79%	21
512	BM25	79%	52%	9
512	Dense	95%	83%	10
512	Hybrid	100%	76%	20
1,012	BM25	76%	52%	9
1,012	Dense	95%	79%	13
1,012	Hybrid	100%	71%	23



Reading. Keyword search alone degrades as look-alikes are added (88% → 76%); dense holds at 95%; the hybrid keeps the right tool among what the model sees **every time**, at about 20 ms. Top-1 slips from 79% to 71%, which is why the model is shown 5 candidates and picks with their descriptions. Limits: 42 questions, template-generated distractors, in-memory Qdrant. Reproduce: `python -m paymind.retrieval.eval_tools`.

12 Scaling to thousands of customers

Today everything runs as one process with SQLite. For thousands of customers the agent design stays the same; the pieces around it become shared services, so any number of stateless API instances can serve any request.

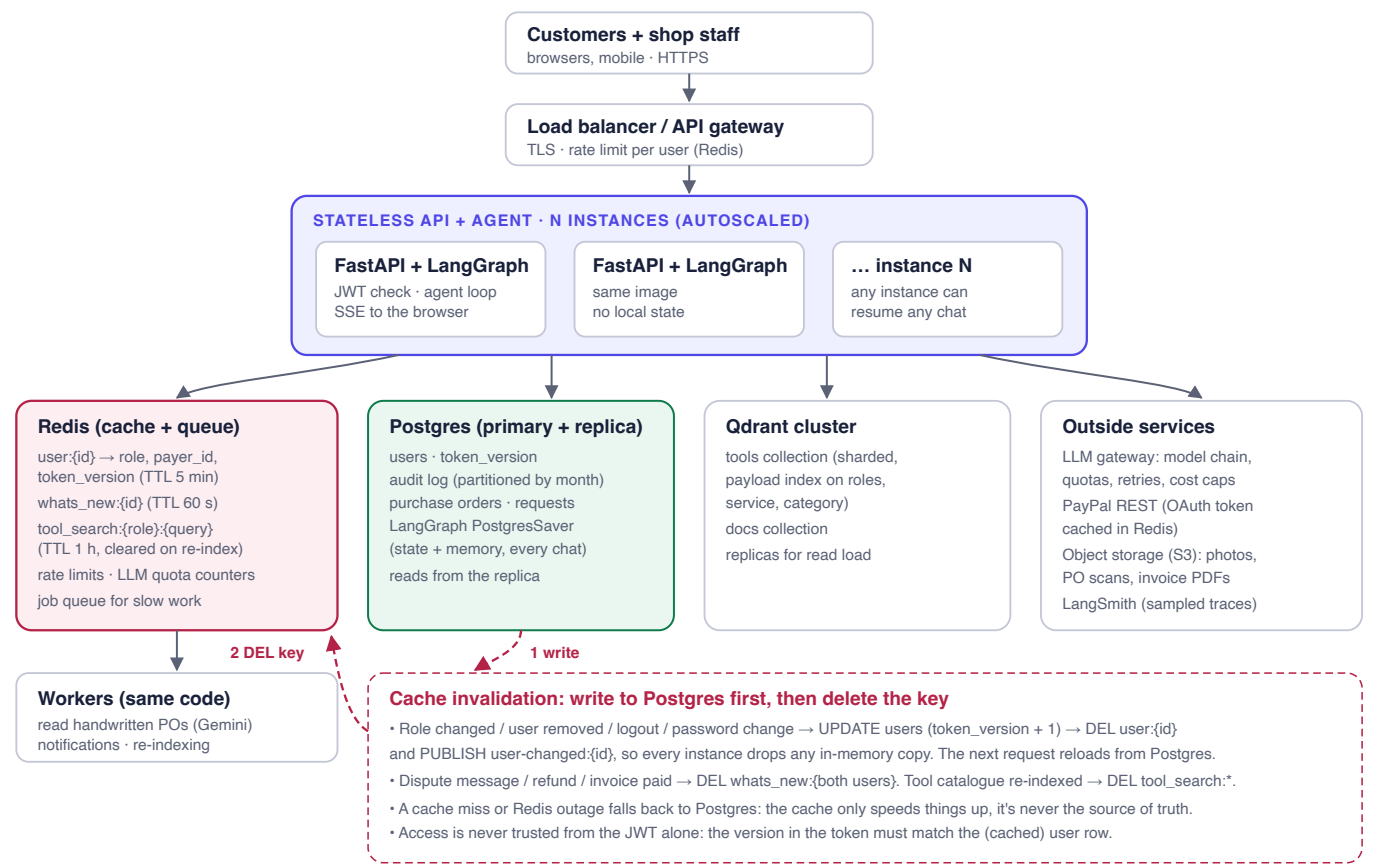


Figure 6. Production architecture. Red: the cache and its invalidation path. The agent graph, tools and rules are unchanged; only where state lives changes.

Pressure	Answer
More users at once	Stateless API instances behind a load balancer; state in Postgres (PostgresSaver), so any instance resumes any chat or confirmation. SSE keeps one connection per answer.
Repeated lookups	Redis: the user row (role, token version) with a short TTL and delete-on-change; What's new per user; tool-search results per role and query. Cache-aside, always falling back to Postgres.
LLM limits and cost	An LLM gateway with per-user rate limits (Redis counters), the existing model fallback chain, and cost caps. One LLM call per round, with a 5-tool prompt.
Slow work	Reading handwritten POs and sending notifications move to a Redis queue and workers, so chat stays fast.
More tools (10,000+)	Same retrieval-first design; shard the Qdrant collection, add a service/category pre-filter and a cross-encoder reranker on the top 20 tools to lift top-1; domain sub-agents only if one prompt can't hold the rules.
Large audit logs	Postgres partitioned by month; LangSmith traces sampled (all errors and writes, a share of reads).

13 Why these choices

Choice	Why	Considered instead
LangGraph	An explicit graph: code decides where the LLM runs, where checks happen and how loops end. Built-in <code>interrupt()</code> for human approval, checkpointing for memory and resume, token streaming, and native LangSmith tracing.	CrewAI / AutoGen: role-play agents, less control per step. A hand-written loop: pause/resume and persistence rebuilt by hand.
Single agent + retrieval	Routing by search costs no LLM call and is measurable (section 11); safety stays in code.	A supervisor with specialist agents: more calls and latency; kept as the growth path.
Qdrant	Sparse (BM25 with IDF) and dense vectors in one collection, server-side RRF, filters inside the search, local mode for tests, managed cloud.	FAISS: dense only, no filters. Chroma: no native sparse + fusion. pgvector: no BM25 fusion built in.
Hybrid search	Keywords catch exact terms and IDs; vectors catch meaning; RRF merges by rank, so no score tuning.	Dense only: 95% top-5 at 1,012 tools vs 100% hybrid. BM25 only: 76%.
fastembed	Small ONNX models run locally (embeddings, BM25, reranker): no API cost, no data leaving the machine, fast.	Hosted embedding APIs.
Gemini	Reliable tool calling, multimodal (handwritten POs), streaming; wrapped in a fallback chain, so the model is swappable.	Any tool-calling model fits the same interface.
LangSmith	Traces LangGraph node by node with no plumbing; <code>@traceable</code> adds our own spans.	Langfuse, OpenTelemetry.
FastAPI + SSE	Typed endpoints, dependency-injected auth, simple one-way streaming for answers.	WebSockets: two-way, not needed.

14 Limitations and next steps

- PayPal is a **mock server** with real request/response shapes; the next step is the PayPal sandbox with OAuth.
- The scaling eval is small (42 questions); grow it from real traces, run it in CI, and add the tool reranker to lift top-1.
- Measure end-to-end task success and tool-choice accuracy with the LLM, not only retrieval.
- Production hardening from section 12: Postgres, Redis, HTTPS, secrets manager, rate limits, object storage.